

LokVirasat Backend & Database Summary

1. What Has Been Created

A complete, production-ready FastAPI backend has been generated to replace the dummy data in the LokVirasat Next.js frontend. The database structure has been set up using PostgreSQL with the PostGIS extension to handle spatial data properly.

Database (PostgreSQL + PostGIS)

We created a fully typed ORM (Object-Relational Mapping) using SQLAlchemy and GeoAlchemy2. The database consists of three main tables:

- **heritage_sites** : Stores fully documented sites. It contains columns for `id` , `name` , `description` , `category` , `verification_status` , and a PostGIS `Geometry("POINT")` column for precise location queries.
- **heritage_leads** : Stores community-submitted tips that need documentation. It contains similar fields but includes `village_or_area` , `submitted_by` , and `status` .
- **site_images** : A relation table to store multiple uploaded images per heritage site.

Backend Application (FastAPI)

The backend codebase is structured inside the `SIH/backend/` directory:

- **main.py** : The entry point of the application. It configures CORS (allowing requests from `localhost:3000`), mounts a static route `/uploads` to serve images, and wires up the API routers.
- **database.py & config.py** : Handles connecting to the local PostgreSQL database using credentials from the `.env` file.
- **models.py** : Defines the SQLAlchemy database tables.
- **schemas.py** : Uses Pydantic v2 to validate incoming API requests and serialize outgoing JSON responses. It automatically converts PostGIS geometry objects into `[longitude, latitude]` arrays for the frontend.
- **crud.py** : Contains helper functions to perform Create, Read, Update, and Delete operations on the database.
- **routers/sites.py & routers/leads.py** : API endpoints defining the REST API:
 - `GET /api/sites/` and `GET /api/leads/`
 - `POST /api/sites/` and `POST /api/leads/`
 - `POST /api/sites/{id}/images` (Supports multipart file uploads)
 - `PATCH /api/leads/{id}/status`
- **seed.py** : A database seeding script pre-filled with 8 real Indian heritage sites (e.g., Chand Baori, Rani ki Vav, Hampi, Dholavira) and 4 heritage leads.

Frontend Integration

The Next.js frontend has been updated to integrate with this new backend:

- The static dummy data imports in `MapPage.tsx` have been removed.
- The map now uses `useEffect` to fetch data from `http://localhost:8000/api/sites/` and `http://localhost:8000/api/leads/` .
- **Bug Fix:** We added deduplication logic in `MapComponent.tsx` to prevent overlapping markers for the same location.

- **Bug Fix:** Map popups have been updated to dynamically inject an `<img>` tag into the leaflet wrapper if the site has images, complete with a smooth CSS fade-in animation.
-

2. What Things Are Remaining

Because you encountered the `externally-managed-environment` error while trying to install Python packages, the virtual environment setup was incomplete from your terminal.

Here is what remains to be done:

1. **Initialize the PostgreSQL database** and create the `lokvirasat` database with the `postgis` extension. (Since you are on Arch Linux, this requires `sudo` privileges).
 2. **Install the Python dependencies** *inside* the virtual environment we created, avoiding the Arch Linux system-wide pip restrictions.
 3. **Run the database seed script** to populate the tables.
 4. **Start the backend server.**
-

3. Commands to Run (Next Steps)

To finish the setup and get everything running, please execute the following commands in your terminal **exactly as written**:

Step A: Start the Database (Requires sudo)

```
# Start and enable PostgreSQL
sudo systemctl start postgresql
sudo systemctl enable postgresql

# Create the database and enable the PostGIS extension
sudo -u postgres psql -c "CREATE DATABASE lokvirasat;"
sudo -u postgres psql -d lokvirasat -c "CREATE EXTENSION IF NOT EXISTS postgis;"
```

(Note: If you have already initialized the database, you can skip to creating the database).

Step B: Install Python Dependencies & Seed Database

You MUST use the virtual environment to install packages. Do not use plain `pip`.

```
# Navigate to the project root
cd /home/harshit/SIH

# Make sure the virtual environment exists
python3 -m venv backend/venv

# Install requirements INSIDE the virtual environment
backend/venv/bin/pip install -r backend/requirements.txt

# Seed the database with the real Indian heritage sites
backend/venv/bin/python -m backend.seed
```

Step C: Run the Servers

Terminal 1: Start the Backend (FastAPI)

```
cd /home/harshit/SIH  
backend/venv/bin/uvicorn backend.main:app --reload --host 0.0.0.0 --port 8000
```

The API will be available at <http://localhost:8000> and the Swagger Docs at <http://localhost:8000/docs>.

Terminal 2: Start the Frontend (Next.js)

```
cd /home/harshit/SIH/LokVirasat  
npm run dev
```

The website will be available at <http://localhost:3000>.